

大模型推理

LARGE LANGUAGE MODEL INFERENCE INFRASTRUCTURE

| 用系统性思维去看推理的 Infra 体系

思考 | 这些技术名词，你认识多少？

- Win32 • Visual Basic • MFC • PHP • JSP • Servlet • ASP.NET
- SQL Server • MVC • SSH • Hadoop • Spark • TensorFlow
- PyTorch • DeepSpeed • vLLM • SGLang

2000 年代 Web 时代 —— 会 SSH (Struts / Spring / Hibernate) 就行

2010 年代 大数据时代 —— 精通 Hadoop / Spark 就行

2020 年代 AI 时代 —— 要精通机器学习框架：TensorFlow / PyTorch / DeepSpeed / vLLM / SGLang

在 AI 时代，技术迭代只会更快 ——
如何保证自己跟得上时代？

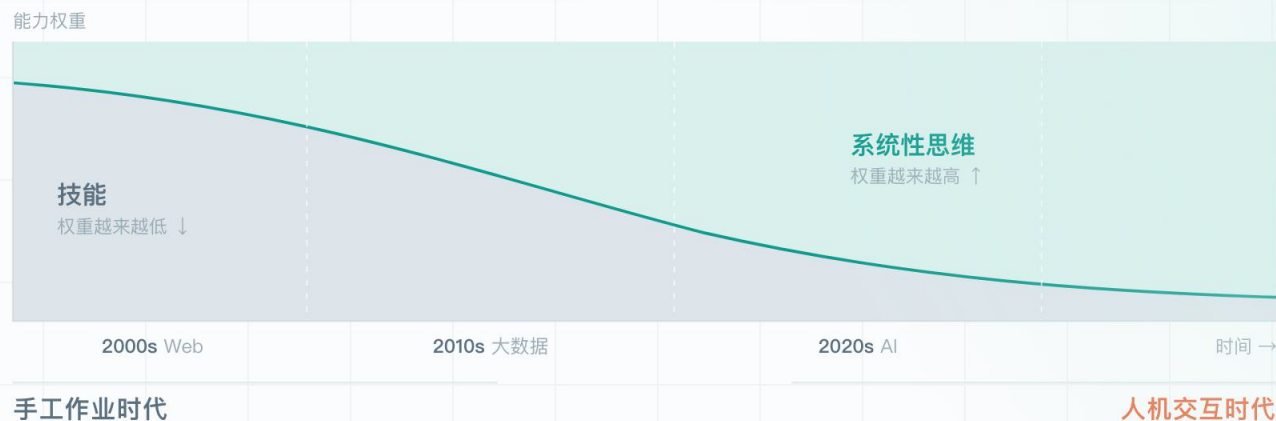
技术能力 = 系统性思维 + 技能

系统性思维

问题的本质是什么？如何拆解，如何分层，边界和取舍在哪里——可迁移，会复利。

技能

具体的语言、框架、API 与工具用法 —— 学得快，也过时得快。



系统性思维的迭代速度 ≪ 技术的迭代速度

系统性思维的第一问 · 拆解问题的本质

所有 Infra 的构建，本质是什么？

以计算范式为出发点

COMPUTE PARADIGM

大数据时代，解决的是单机处理大量数据的瓶颈问题，所以才有了 **MapReduce** 范式，才有了 Hadoop、Spark、Yarn 等产品；大规模并行带来了运维成本，以及互联网爆发带动的云的发展，衍化出了 **Kubernetes**。



MAPREDUCE · 计算范式

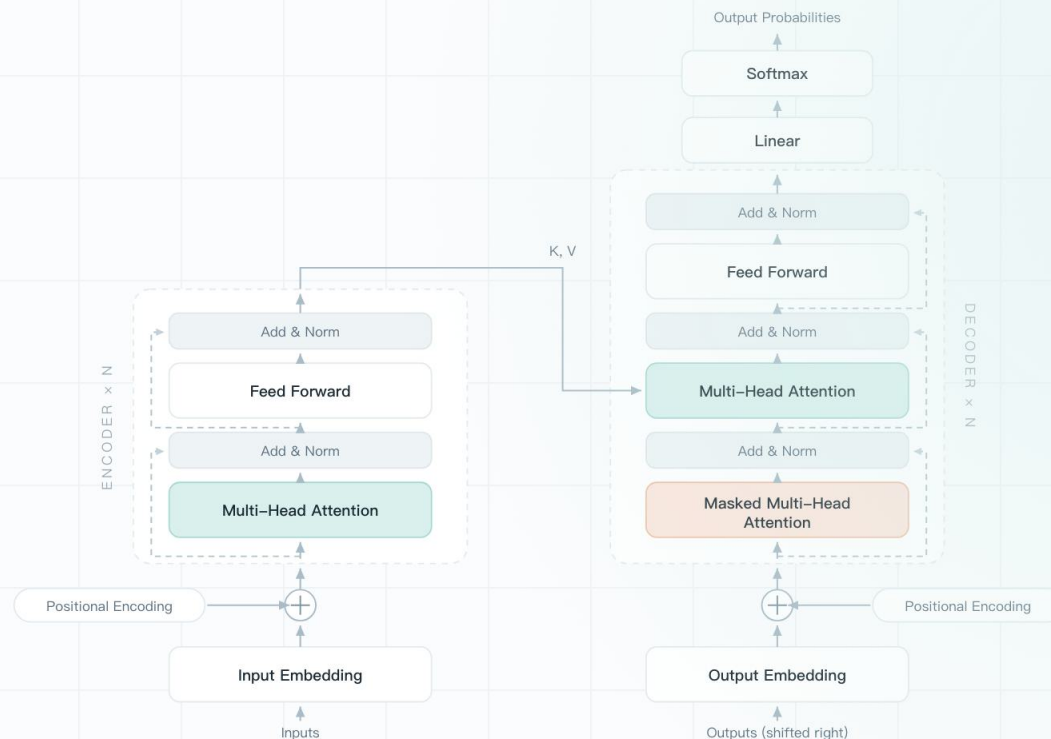
那在 AI 时代，大模型推理的 Infra 应该是什么样的？

大模型推理的**计算范式**，围绕的是**模型的结构**；而模型结构，几乎都是以**Transformer**为基础衍化出来的。

1 大量的矩阵计算

Attention 与 FFN 本质是稠密矩阵乘法，CPU 扛不住 —— 必须靠 GPU 的并行算力。

COMPUTE · GPU



2 线上大规模推理，单节点不够

模型放不下、请求扛不住，如何横向扩展 —— 分布式并行与集群调度。

SCALABILITY

3 GPU 很贵，效率怎么提高

算力昂贵且常被显存与带宽拖住，如何把每一张卡的利用率压榨到极致。

EFFICIENCY · COST

系统性思维的第二问 · 如何分层

承接 ① 大量的矩阵计算 → 如何解决

算力与计算，两层承接

底层靠 GPU 把算力供出来，上层靠推理框架把计算排布好 —— 框架做的所有事，都指向同一个目标：结合计算本身的特征，尽可能榨干每一张卡。



同一批请求 · GPU 的时间线

只有 GPU

大量空泡

算子零散 · 显存碎片 · 请求排队 —— 卡在等

GPU + 框架

几乎不留空

算子融合 · 显存分页 · 连续批处理 —— 卡在算

1 Flash Attention

把 attention 分块放进片上 SRAM 里算，不再把巨大的中间矩阵写回显存：算的是同一件事，省掉的是搬运。

2 Paged Attention

把 KV Cache 切成定长 block 分页管理，消除显存碎片：同样一张卡，能同时装下更多并发请求。

3 Continuous Batching

不等整批做完，谁生成结束谁先下车、新请求立刻补位，GPU 不再为了等齐一批而空转。

系统性思维的第二问 · 如何分层

承接 ① 大量的矩阵计算 → GPU 怎么部署

GPU 部署，解决算力

这一块在云原生的环境里，依托 **Kubernetes**，要提供几个核心能力：



异构 GPU

HETEROGENEOUS

集群里混跑不同型号、不同代次的卡，统一纳管、按算力与显存画像调度。



跨数据中心部署

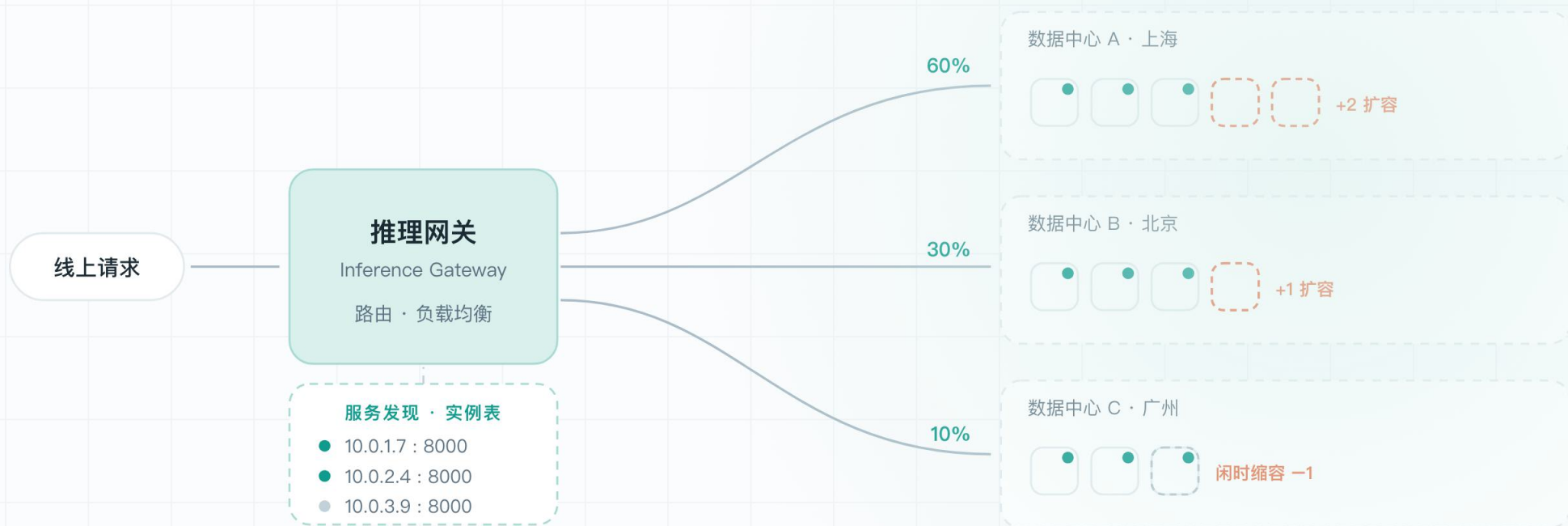
MULTI-DC / MULTI-CLOUD

算力散落在多地机房与多云，需要统一编排、就近接入与跨地域容灾。

系统性思维的第二问 · 如何分层

承接 ② 单节点不够 → 可扩展性

大规模推理，怎么扩展



1 多数据中心流量分配

按就近时延、机房容量与权重把请求切到不同机房与云，兼顾体验与跨地域容灾。

2 算力调度层 · 弹性扩缩容

依据 QPS、排队时延与 GPU 利用率自动增减副本：忙时补齐，闲时释放昂贵的卡。

3 推理网关 · 服务发现

实例注册与健康检查，按模型、版本、租户路由；后端副本增减对调用方完全透明。

系统性思维的第二问 · 如何分层

承接 ③ GPU 很贵 → 计算效率

计算效率，靠组件协同

效率不是某一层的单点优化，而是**框架 · 网关 · 模型**三层一起把算力省下来。

框架层

VLLM / SGLANG

Flash Attention

解决访存瓶颈 —— 分块计算 + 算子融合，避免把巨大的注意力矩阵反复读写 HBM。

Paged Attention

解决显存碎片 —— 像操作系统分页一样管理 KV 显存，按需分配、显存利用率拉满。

KV Cache

解决计算复用 —— 已算过的 K/V 存下来，解码每一步不必重算历史 token。

...

网关层

GATEWAY

KV Cache 感知

与框架实例协同 —— 网关掌握每个实例缓存了哪些前缀，把缓存状态变成可调度的信息。

缓存友好路由

大模型特有的调度 —— 同前缀的请求打到已命中的实例，直接省掉重复的 prefill 计算。

模型层

MODEL

微调 · 蒸馏

用更小的模型顶上 —— 在具体场景上让小模型达到大模型同等能力，单请求成本直接下一个量级。

量化 · 剪枝

让计算本身变少 —— 降低权重与激活精度、裁掉冗余结构，省算力也省显存带宽。

整体架构，各层怎么协同



算力靠 **Kubernetes** 拿到，扩展性靠 **推理网关** 做掉，效率靠 **框架 · 网关 · 模型** 一起省。

系统性思维的核心，是一套工程方法

以计算范式为出发点，解决效率 · 可扩展性 · 组件协同的系统工程方法。



效率

EFFICIENCY

范式决定算力怎么花：稠密矩阵乘必须落到 GPU，KV Cache 决定显存与带宽才是真瓶颈。



可扩展性

SCALABILITY

单点算不完就横向摊开：网关做服务发现与流量分配，调度层做副本编排与弹性扩缩容。



组件协同

COORDINATION

没有哪一层能单独把效率做出来：框架 · 网关 · 模型分层各司其职，再合起来省算力。

技术名词一直在换，但「先问计算范式是什么」这一步不会变。

系统性思维 · 延展

以 KV Cache 为例

同一个技术名词，你能想到第几层

技术会换，但追问的层次不会变 —— 换成任何一个新名词，这三问都成立。

LEVEL 01

知其然

WHAT

解决什么问题

自回归解码每生成一个 token，都要重算全部历史 token 的 K/V，长度一长就是 $O(n^2)$ 的重复计算。

为什么需要它

把算过的 K/V 缓存下来，每步只算新 token —— 用显存换算力，decode 回到线性复杂度。

边界在哪里

只对 decode 阶段有效，prefill 仍然是算力密集；缓存命中，就退化回重算。

LEVEL 02

知其所以然

HOW

怎么存、怎么索引

vLLM 用 PagedAttention 把 KV 切成定长 block，靠 block table 做逻辑到物理的映射，像虚拟内存一样消除碎片。

粒度怎么控

block 太大，内部碎片白占显存；block 太小，block table 变长，索引与 kernel 开销跟着上来。

怎么权衡

显存占用 · 碎片率 · 前缀命中率，还要和 batch size、吞吐与时延一起算总账。

LEVEL 03

知其所以必然

WHAT'S NEXT

放不下

长上下文与多轮 Agent 让 KV 规模爆炸，需要 HBM / DRAM / SSD 分层存储与卸载换入。

要复用

从单实例走向全局前缀缓存：跨请求、跨实例共享，配套 KV 感知路由与失效一致性。

要搬运

P/D 分离后 KV 要跨节点传输，RDMA 带宽与传输时延成为新的瓶颈；压缩与量化又要换精度。

停在第一层，是技术的使用者；走到第三层，才是 Infra 的建设者。